Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

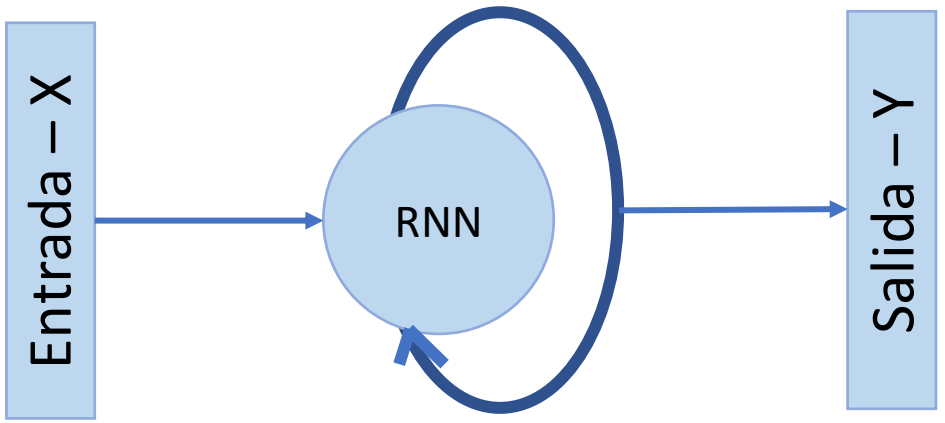
Clase # 16 - PAO 2026 – II

Agenda

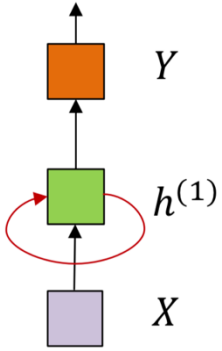
1. Redes neuronales recurrentes – RNN: Continuación
   1.1 Un ejemplo
   1.2 Backprop through time - BPTT
2. Desvanecimiento del gradiente
3. Introducción a las redes Long Short Term Memory – LSTM

Redes neuronales recurrentes

- Veamos un ejemplo: RNN, para predecir caracteres.
- Supongamos que tenemos la cadena de caracteres “aire”; y, luego de ingresar las primeras 3 letras: “a, i, r”, deseamos que la RNN prediga la siguiente letra: “e”.
- En este ejemplo el vocabulario tiene 4 letras: {a, i, r, e}, y el algoritmo fue entrenado, recordar que este vocabulario se denomina “Corpus”.
- Veamos cómo esta estructura se utiliza para predecir la siguiente letra en la palabra “aire”.



Redes neuronales recurrentes

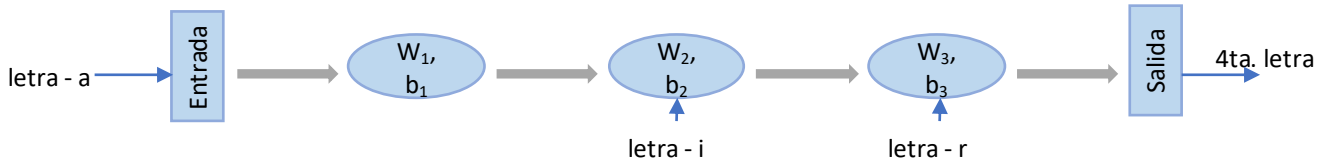


Procedimiento para el entrenamiento:

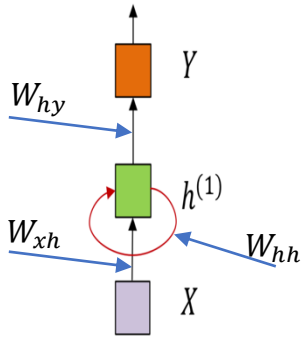
1. Ingresamos un símbolo a la vez (un time step); es decir, un X_t se suministra a la red.
2. Luego calculamos su estado actual utilizando la combinación de la entrada actual y el estado anterior, es decir calculamos h_t. Este h_t actual se convierte luego en h_{t-1} para el siguiente paso de tiempo.
3. Ejecutar tantos pasos de tiempo como el problema lo requiera y combinar la información de todos los estados anteriores.
4. Una vez que se completan todos los pasos de la secuencia, utilizamos el estado actual final para calcular la salida Y_t.
5. La salida se compara con la salida deseada y se calcula el error.
6. Luego, el error se propaga hacia atrás de la red para actualizar los pesos... Al final la red está entrenada.

Redes neuronales recurrentes

- Como en los casos anteriores, primero calculamos la salida, esto es el paso forward: Ingresamos las letras en secuencia:
  - En este ejemplo, ingresa ”a", la cual no tiene nada que la preceda, calculamos h_0.
  - Luego ingresamos la letra ”i”.
  - En el momento que suministramos la letra ”i", se aplica la fórmula de recurrencia a esa letra, y al estado anterior de la capa recurrente, que fue la letra ”a”.
  - Este proceso de ingresar cada letra, se lo conoce como pasos de tiempo o time steps; en este ejemplo necesitamos 3 time steps, para determinar la salida.
  - Si en el momento t, la entrada es ”i", en el tiempo $t-1$, la entrada fue ”a”.
  - Para obtener el nuevo estado, la fórmula de recurrencia se aplica tanto a “i” como “a”, así:



Redes neuronales recurrentes



- Recordar siempre: $y = f(\mathrm{Z})$; $\mathrm{Z} = \sum \mathrm{WX}$
- $Z_0 = \sum(W_{xh}X_0 + W_{hh}h_{-1})$
- $h_0 = f_1(Z_0)$
- h_0 es el nuevo estado, h_{-1} es el estado anterior, y X_0 es la entrada actual.
- Supongamos que la función de activación f_1 es *tanh*; que los pesos en la capa recurrente son W_{hh}; y, que los pesos en la neurona de entrada son W_{xh}.
- Entonces la ecuación que determina el estado h_0 en el momento $t = 0$ es:

$$h_0 = tanh(W_{xh}X_0 + W_{hh}h_{-1} + bias_h)$$

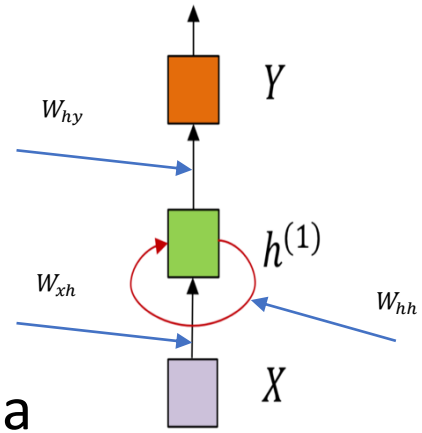
Redes neuronales recurrentes

- Veamos el paso “forward” en nuestro ejemplo:

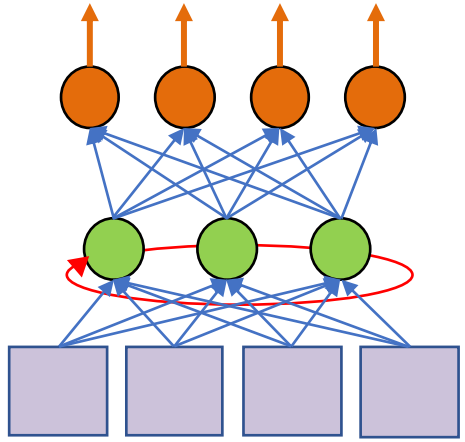
a		i		r		e
1		0		0		0
0		1		0		0
0		0		1		0
0		0		0		1

- Para simplificar el ejemplo, supongamos que el embedding de las letras están en formato: one-hot-encoded
- El vocabulario completo es: {a, i, r, e}

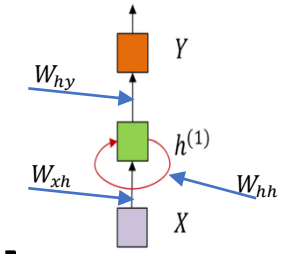
Redes neuronales recurrentes



- Para este problema supongamos el siguiente diseño de la RNN:
  - La ventana es de 3 time-steps (3 letras en secuencia para predecir la cuarta letra)
  - El vector de entrada es de tamaño 4, porque usamos one-hot
  - Definamos una capa oculta con 3 neuronas de ancho
  - Entonces, para este problema necesitamos tres matrices W:
    1) $\mathrm{W_{xh}} \in \mathbb{R}^{3\times4}$ (3 neuronas x 4 time-steps + bias)
    2) $\mathrm{W_{hh}} \in \mathbb{R}^{3\times3}$ (3 neuronas x 3 neuronas + bias)
    3) $\mathrm{W_{hy}} \in \mathbb{R}^{4\times3}$ (3 neuronas x 4 neuronas + bias)
  - El bias de la capa oculta, y de la capa de salida
    $b_h \in \mathbb{R}^{3\times1}$
    $b_y \in \mathbb{R}^{4\times1}$



Redes neuronales recurrentes



- Para la letra “a”, como no tenemos un estado previo, el estado es [0, 0, 0], entonces el estado actual, en el paso 0, es:

$$h_0 = tanh(W_{xh}X_0 + (W_{hh}h_{0-1} + bias_h))$$

- Veamos el cálculo: $W_{hh}h_{-1} + bias_h$

W_{hh}		
0.1	0. 2	−0.1
−0.1	0.1	0.3
0.2	−0. 2	0.1

$\times$

h_{-1}
0
0
0

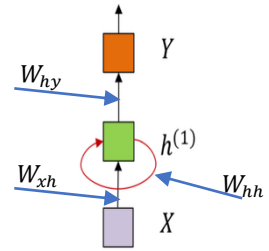
$+$

$bias_h$
0.2
0.1
0

$=$

$W_{hh}h_{0-1} + bias_h$
0.2
0.1
0

Redes neuronales recurrentes

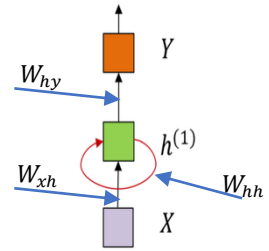


- Entonces: $h_0 = tanh(W_{xh}X_0 + (W_{hh}h_{0-1} + bias_h))$, es:

$h_0 = \tanh \left\{ \begin{array}{|c|}\hline W_{xh}X_0 \\ \hline 0.2 \\ \hline 0.1 \\ \hline -0.3 \\ \hline \end{array} + \begin{array}{|c|}\hline W_{hh}h_{0-1} + bias_h \\ \hline 0.2 \\ \hline 0.1 \\ \hline 0 \\ \hline \end{array} \right\}$

$h_0 = \begin{array}{|c|}\hline h_0 \\ \hline 0.3799 \\ \hline 0.1974 \\ \hline -0.2913 \\ \hline \end{array}$

Redes neuronales recurrentes



- Luego en el próximo paso de tiempo $t = 1$, “i” ingresa en la red.
- El estado h_0 del cálculo anterior se convierte en h_{t-1}.
- La letra “i” one-hot encoded ahora es X_1, con estos valores, calculemos el nuevo estado h_1.

$$h_1 = tanh((W_{hh}h_0 + bias_h) + W_{xh}X_1)$$

- El cálculo: $W_{hh}h_0 + bias_h$ es:

$W_{hh}h_0 + bias_h =$

W_{hh}		
0.1	0. 2	−0.1
−0.1	0.1	0.3
0.2	−0. 2	0.1

$\times$

h_0
0.3799
0.1974
−0.2913

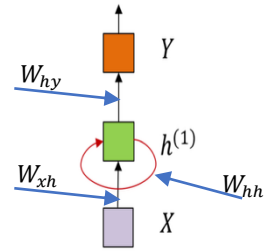
$+$

$bias_h$
0.2
0.1
0

$=$

$W_{hh}h_0 + bias_h$
0.3066
−0.0057
0.0074

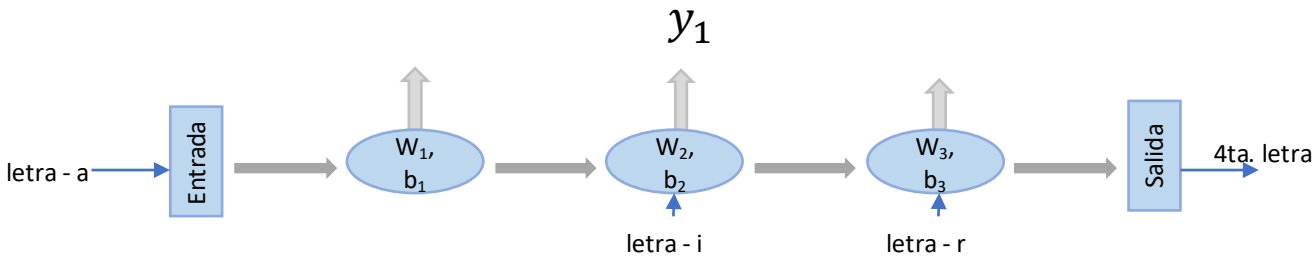
Redes neuronales recurrentes



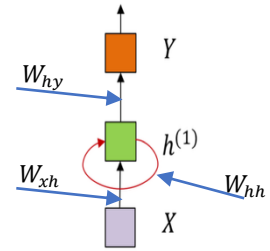
- Entonces el cálculo de $h_1 = tanh(W_{hh}h_0 + W_{xh}X_1)$, es:

$h_1 = \tanh\left\{ \begin{array}{|c|} \hline \boldsymbol{W_{hh}h_0 + bias_h} \\ \hline 0.3066 \\ \hline -0.0057 \\ \hline 0.0074 \\ \hline \end{array} + \begin{array}{|c|} \hline \boldsymbol{W_{xh}X_1} \\ \hline -0.1 \\ \hline 0.3 \\ \hline 0.2 \\ \hline \end{array} \right\}$

$h_1 = \begin{array}{|c|} \hline \boldsymbol{h_1} \\ \hline 0.2037 \\ \hline 0.2861 \\ \hline 0.2045 \\ \hline \end{array}$



Redes neuronales recurrentes



- La matriz W_{hy} es de dimensión 4x3 (4 neuronas en la salida y 3 neuronas que se conectan desde la capa oculta) + $bias_y$

W_{hy}		
0.2	−0.1	0.3
0.0	0.4	−0.2
−0.3	0.1	0.2
0.1	−0. 2	0.0

$\times$

h_1
0.2037
0.2861
0.2045

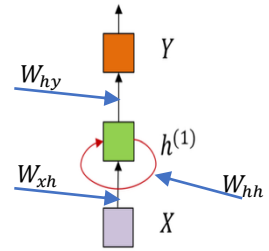
$+$

$bias_y$
0
-0.1
0.4
0.3

$=$

$W_{hy}h_1 + bias_y$
0.0735
−0.0264
0.4084
0.2631

Redes neuronales recurrentes



- Finalmente, calculamos la probabilidad de las letras en el vocabulario, por ejemplo, aplicando la función de umbral softmax.
- La letra en particular que el sistema recomendaría, en nuestro ejemplo, después de “i”, sería la letra que obtenga la mayor probabilidad, luego de aplicar softmax.
- En este ejemplo la “r”:

$$Y_1 = \text{Softmax}$$

$W_{hy}h_1 + bias_y$
0.0735
−0.0264
0.4084
0.2631

$=$

y_1
0.2217
0.2006
0.3098
0.2679

Redes neuronales recurrentes

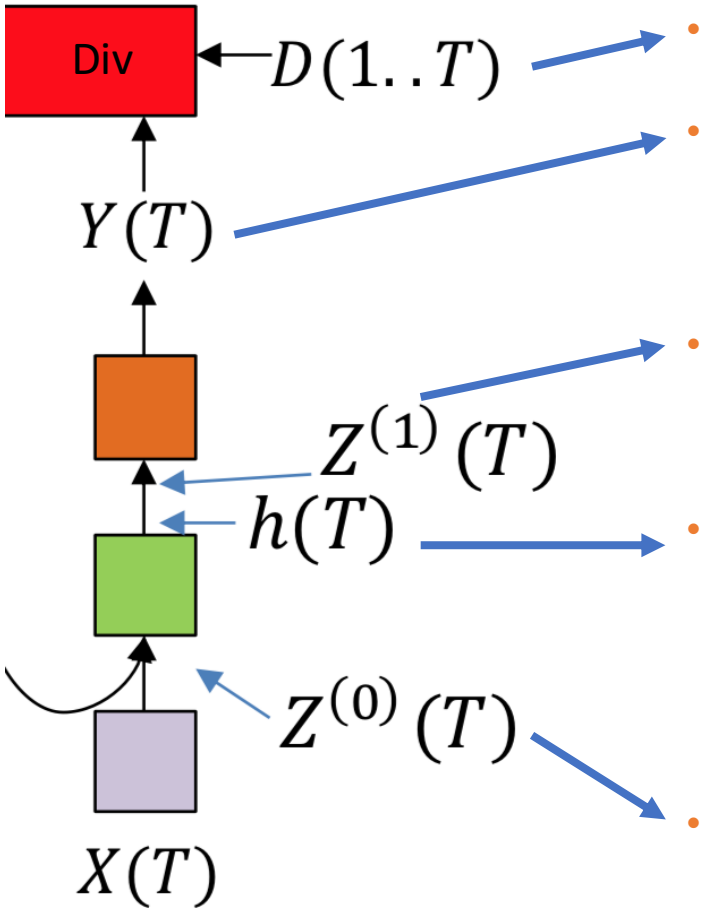
- Estas probabilidades lo que nos dicen es que la letra después de ”i” debe ser “r”, que tiene la probabilidad más alta.
- Aunque la predicción es correcta, recordar que apenas hemos empezado a entrenar la red.
- Acabamos de mostrarle dos letras. Así que prácticamente no ha aprendido nada todavía.
- Una vez que determine la última letra “e”, cómo se actualizan los pesos?
- Igual que los modelos anteriores, las redes RNN “aprenden” también a través de backpropagation.
- En RNN este procedimiento se llama “Back Propagation Through Time” (BPTT).

Redes neuronales recurrentes

- Procedimiento para el algoritmo BPTT:

1. Calculamos el error utilizando la salida actual y la salida esperada.
2. Desenrollamos todos los pasos de tiempo.
3. Para la red desenrollada, se calcula el gradiente para cada paso de tiempo con respecto a los pesos.
4. En el paso forward los pesos son los mismos para todos los pasos de tiempo, en backprop los gradientes se actualizan en cada paso.
5. Los pesos se actualizan para las neuronas recurrentes y las capas de salida.

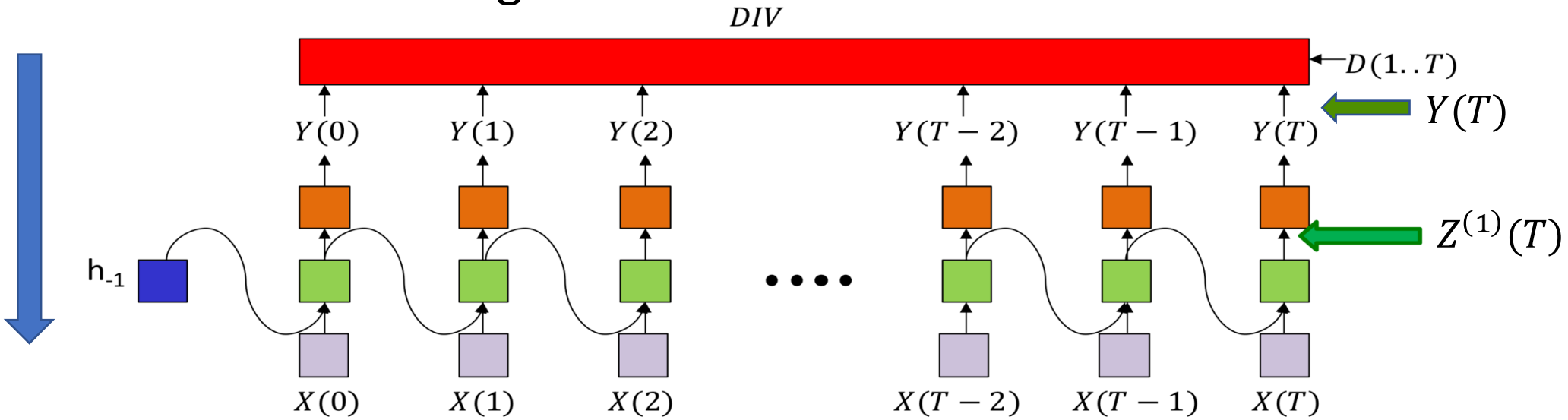
Redes neuronales recurrentes



- D(1..T) es el valor deseado en el time step T
- $Y(T)$ es la salida de la red en el paso del tiempo T
  - $Y_i(t)$ es la salida i^{th} en time step t
- $Z^{(1)}(T)$ es el valor previo a la activación de las neuronas en la capa 1, en el time step T
- $h(T)$ es la salida de la capa oculta en el tiempo T
  - En este diagrama sólo tenemos una capa escondida
- $Z^{(0)}(T)$ es el valor previo a la activación de las neuronas de la primera capa oculta, al tiempo T

Redes neuronales recurrentes

- Cómo se calcula el gradiente en BPTT

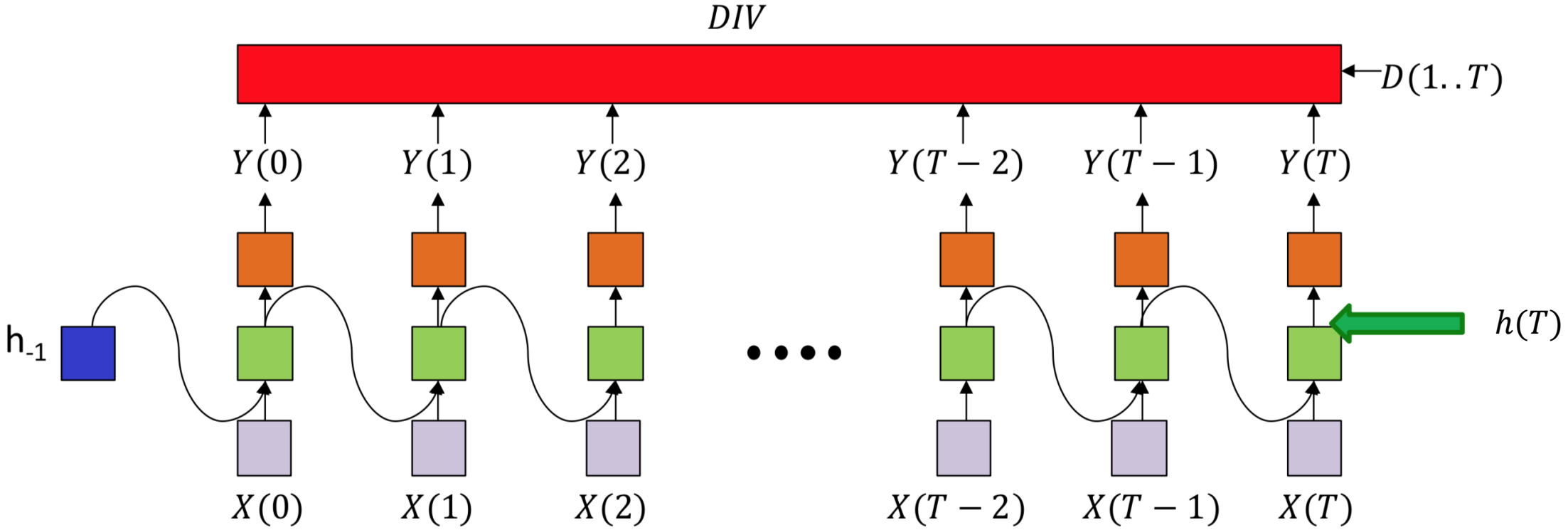


- Primer paso (Inicializar el gradiente): Calcular $\frac{dDIV}{dY_i(T)}$ para todas las i
- Luego: $\frac{dDIV}{dZ_i^{(1)}(T)} = \frac{dDIV}{dY_i(T)} \frac{dY_i(T)}{dZ_i^{(1)}(T)}$
- o el vector: $\frac{dDIV}{dZ_i(T)} = \sum_i \frac{dDIV}{dY_i(T)} \frac{dY_i(T)}{dZ_i^{(1)}(T)}$

Sumatoria sobre las **neuronas de salida** o **clases** en el time-step T

Redes neuronales recurrentes

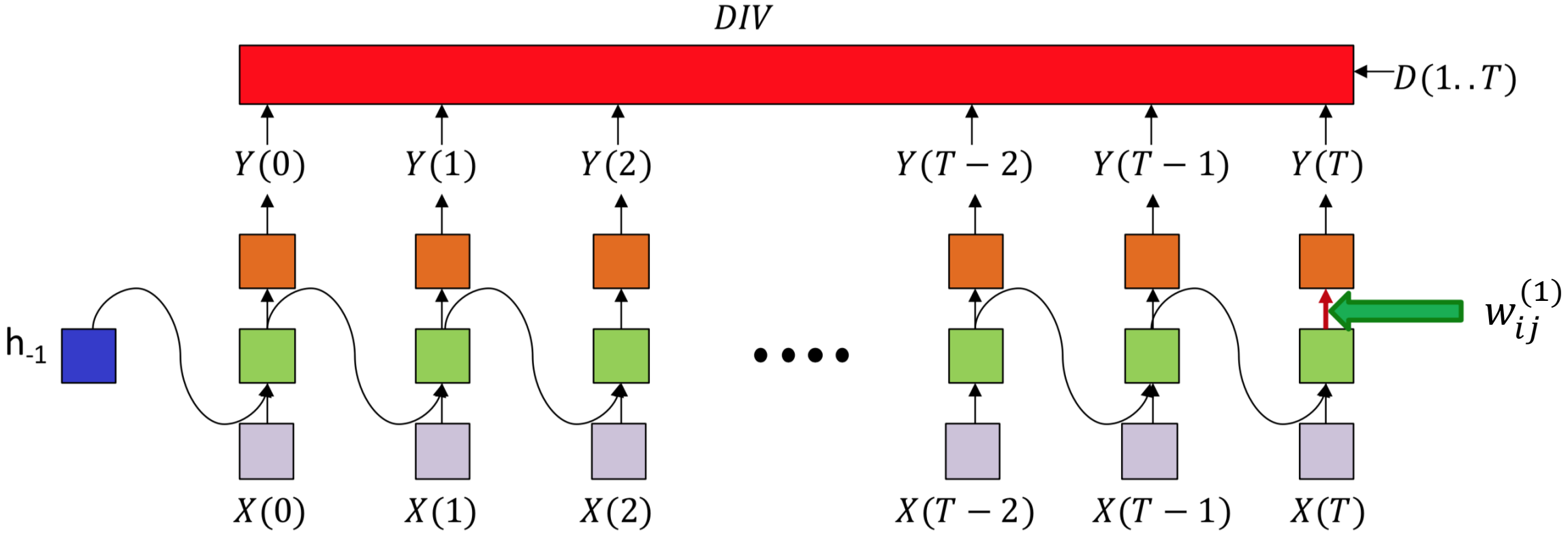
- Cómo se calcula el gradiente en BPTT



- $$\frac{dDIV}{dh_i(T)} = \sum_j \frac{dDIV}{dZ_j^{(1)}(T)} \frac{dZ_j^{(1)}(T)}{dh_i(T)} = \sum_j w_{ij}^{(1)} \frac{dDIV}{dZ_j^{(1)}(T)}$$

Redes neuronales recurrentes

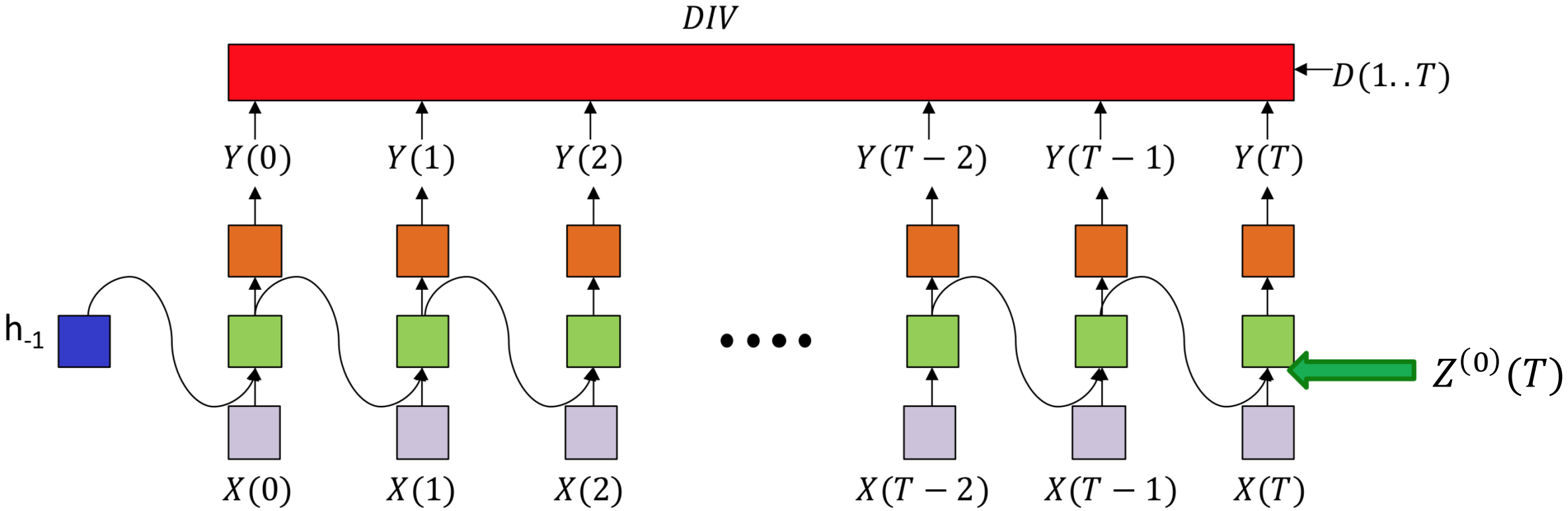
- Cómo se calcula el gradiente en BPTT



- $$\frac{dDIV}{dw_{ij}^{(1)}} = \frac{dDIV}{dZ_j^{(1)}(T)} \frac{dZ_j^{(1)}(T)}{dw_i(T)} = \frac{dDIV}{dZ_j^{(1)}(T)} h_i(T)$$

Redes neuronales recurrentes

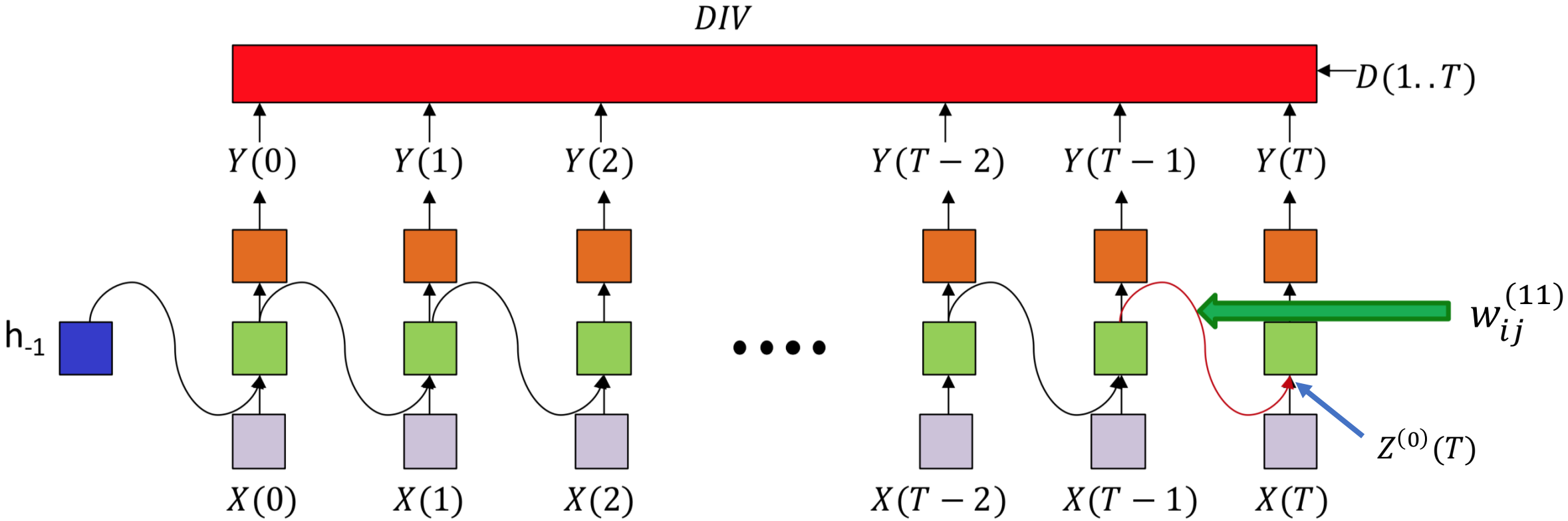
- Cómo se calcula el gradiente en BPTT



- $$\frac{dDIV}{dZ_i^{(0)}(T)} = \frac{dDIV}{dh_i(T)} \frac{dh_i(T)}{dZ_i^{(0)}(T)}$$

Redes neuronales recurrentes

- Cómo se calcula el gradiente en BPTT

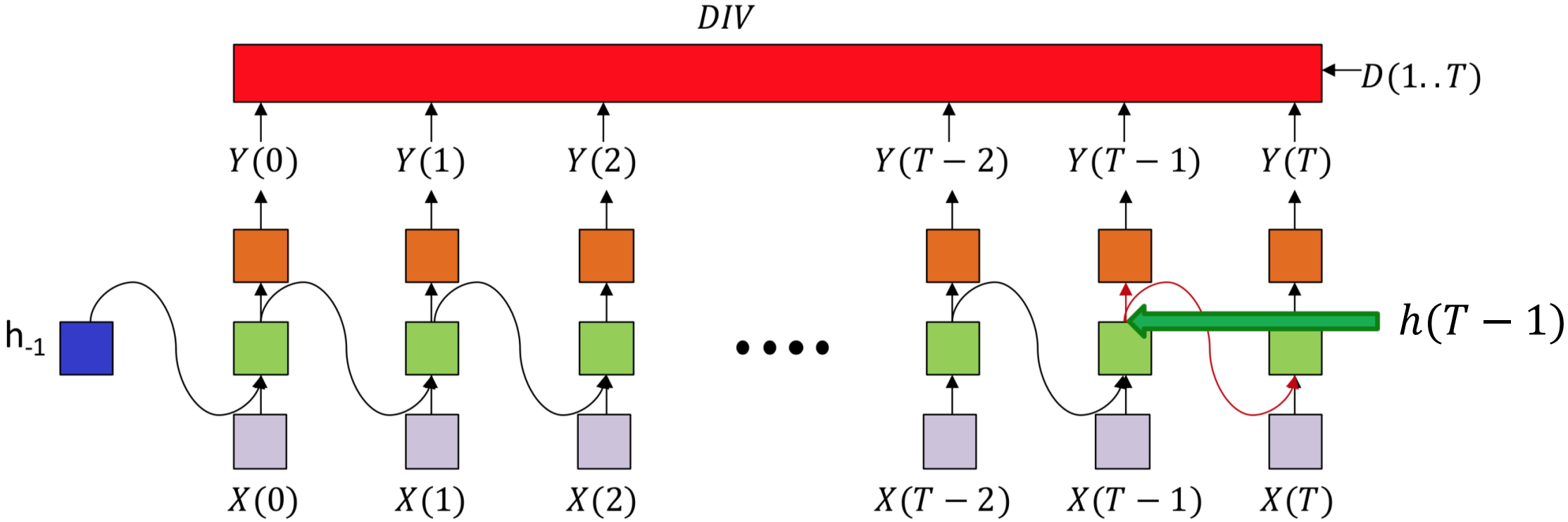


- $\frac{dDIV}{dw_{ij}^{(11)}} = \frac{dDIV}{dZ_j^{(0)}(T)} h_i(T-1)$

$$z = \sum W h_i$$

Redes neuronales recurrentes

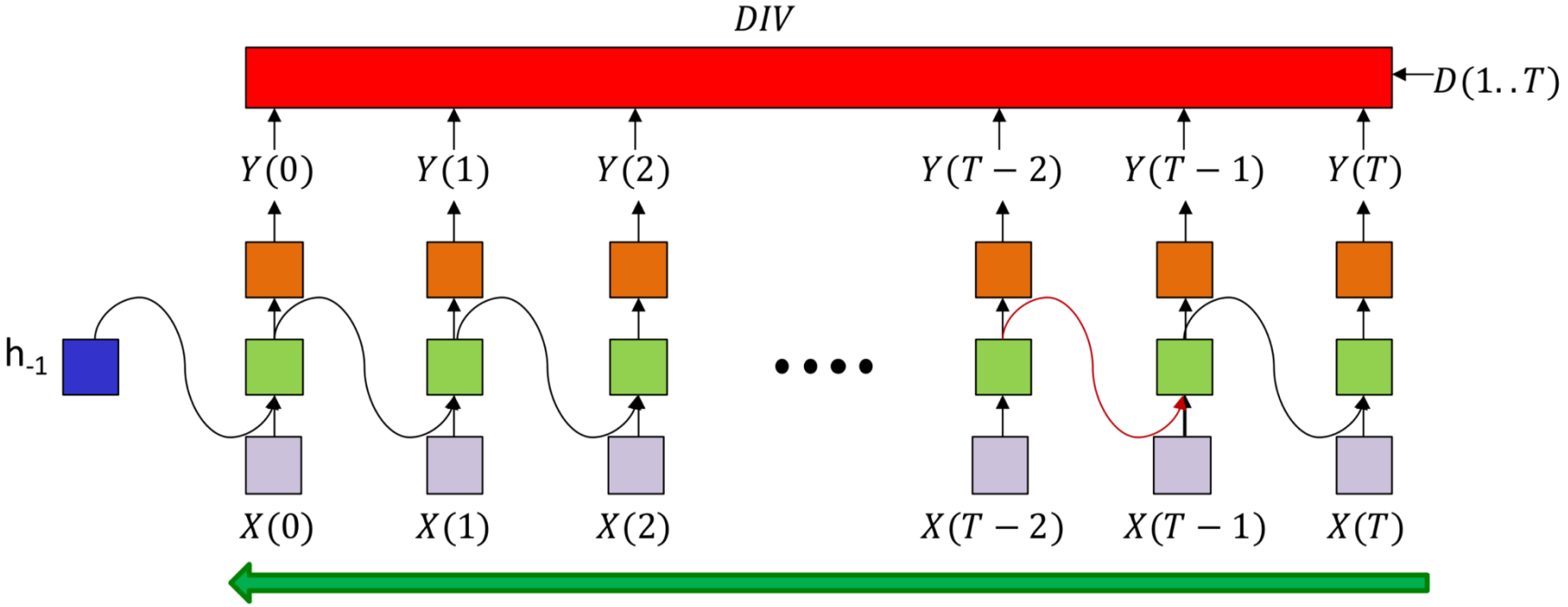
- Cómo se calcula el gradiente en BPTT



- $$\frac{dDIV}{dh_i\,(T-1)} = \sum_j w_{ij}^{(1)} \frac{dDIV}{dZ_j^{(1)}\,(T-1)} + \sum_j w_{ij}^{(11)} \frac{dDIV}{dZ_j^{(0)}(T)}$$

Redes neuronales recurrentes

- Cómo se calcula el gradiente en BPTT



$$\frac{dDIV}{dh_{-1}} = \sum_j w_{ij}^{(11)} \frac{dDIV}{dZ_j^{(1)}(0)}$$

$$\frac{dDIV}{dw_{ij}^{(0)}} = \sum_t \frac{dDIV}{dZ_j^{(0)}(t)} X_i(t)$$

$$\frac{dDIV}{dw_{ij}^{(11)}} = \sum_t \frac{dDIV}{dZ_j^{(0)}(t)} h_i(t-1)$$

Redes neuronales recurrentes

- En el ejemplo, para calcular el error $t = 3$, o el cuarto paso de tiempo, con respecto al primero tenemos:

$$\frac{\partial E}{\partial W} = \frac{\partial E}{\partial Y_3} * \frac{\partial Y_3}{\partial h_3} * \frac{\partial h_3}{\partial Y_2} * \frac{\partial Y_2}{\partial h_1} \ldots$$ hasta llegar a $W^{(0)}$

- Vemos que hay una larga dependencia.
- Al aplicar la regla de la cadena, si cualquiera de los gradientes se acerca a 0, entonces todos los gradientes se acercarán a 0 exponencialmente de manera rápida, debido a la multiplicación.
- Esos estados ya no ayudarían a la red a aprender nada.
- Esto se conoce como el problema de **desvanecimiento del gradiente.**

Long Short Term Memory Networks - LSTM

- Las redes Long Short Term Memory - LSTM, resuelven este problema, son un tipo especial de RNN.
- Son capaces de aprender dependencias a largo plazo.
- Fueron propuestas por Hochreiter y Schmidhuber en 1997.
- Funcionan bien en una gran variedad de problemas.
- Las LSTM también tienen esta estructura tipo secuencial, pero el módulo recurrente tiene una estructura diferente; en lugar de tener una sola capa de red neuronal, tienen múltiples capas, que interactúan de una manera particular.
- Las LSTM tienen 3 puertas: Una de entrada, una de olvido y una de salida.

Long Short Term Memory Networks - LSTM

- Arquitectura de las LSTM

